

```
# Documento Técnico – Vec3D
```

```
**Asignatura:** Compiladores
**Proyecto:** DSL Estudiante 6 – Vec3D (Vectores tridimensionales)
**Herramienta:** Python 3 (lexer + parser descendente recursivo)
**Extensión:** `.vec3d`
```

```
---
```

1. Descripción del DSL

Vec3D es un lenguaje para operar con vectores de tres componentes. Permite declarar vectores, calcular producto punto, producto cruz y norma, y combinar vectores con operaciones aritméticas básicas.

Ejemplo:

```
```text
vec a = (1, 2, 3);
vec b = (4, 5, 6);
imprimir dot(a, b);
vec c = a + b;
imprimir c;
```
```

```
---
```

2. Tabla de tokens

| Token | Regla | Ejemplo |
|---------------|---------------------------|---------------|
| `vec` | palabra reservada | `vec` |
| `dot` | palabra reservada | `dot` |
| `cross` | palabra reservada | `cross` |
| `norm` | palabra reservada | `norm` |
| `imprimir` | palabra reservada | `imprimir` |
| `var` | palabra reservada | `var` |
| `si` / `sino` | palabra reservada | `si` |
| `ID` | `[a-zA-Z][a-zA-Z0-9_]*` | `a`, `umbral` |
| `INT` | `-[0-9]+` | `3`, `-1` |
| `FLOAT` | `-[0-9]+\.[0-9]+` | `3.14` |
| `STRING` | `"...` | `"texto"` |
| Operadores | `+ - * / < > <= >= == !=` | `+`, `==` |
| Delimitadores | `( ) { } ; , =` | `;` |
| Comentarios | `//` y `/* */` | `// nota` |

Errores léxicos reportan línea y columna.

```
---
```

3. Gramática EBNF

```
```ebnf
program = statement_list ;
```

```

statement = vec_decl | var_decl | assignment | print_stmt | if_stmt
;

vec_decl = "vec" ID "=" expr ";" ;
var_decl = "var" ID ["=" expr] ";" ;
assignment = ID "=" expr ";" ;
print_stmt = "imprimir" expr ";" ;
if_stmt = "si" "(" expr ")" block ["sino" block] ;
block = "{" statement_list "}" ;

expr = relational ;
relational = additive { ("<" | ">" | "<=" | ">=" | "==" | "!=") additive } ;
additive = multiplicative { ("+" | "-") multiplicative } ;
multiplicative = unary { ("*" | "/") unary } ;
unary = ["+" | "-"] primary ;
primary = FLOAT | INT | STRING | ID | vector_literal
 | dot_call | cross_call | norm_call | "(" expr ")" ;

vector_literal = "(" expr { "," expr } ")" ;
dot_call = "dot" "(" ID "," ID ")" ;
cross_call = "cross" "(" ID "," ID ")" ;
norm_call = "norm" "(" ID ")" ;
\`\`\`

```

Precedencia: relacional → aditivo → multiplicativo → unario.

---

#### ## 4. Reglas semánticas

##### ### Generales

- Variables no declaradas, redeclaradas o usadas antes de asignación
- Tabla de símbolos con ámbitos en bloques `si`/`sino`
- Chequeo de tipos en expresiones y asignaciones

##### ### Del dominio Vec3D

**\*\*R1 – Dimensión fija 3:\*\*** todo vector debe tener exactamente 3 componentes.

```

\`\`\`text
vec a = (1, 2);
\`\`\`

```

Error: `el vector 'a' tiene 2 componentes; se esperaban exactamente 3`

**\*\*R2 – Tipos de retorno:\*\*** `dot` y `norm` devuelven escalar; `cross` devuelve vector. No se puede sumar escalar con vector.

**\*\*R3 – Multiplicación:\*\*** `3 \* a` es válido; `a \* b` es error (ambiguo entre dot y cross).

---

## ## 5. AST

Nodos principales: `Program`, `VecDecl`, `VarDecl`, `VectorLiteral`, `DotCall`, `CrossCall`, `NormCall`, `BinaryOp`, `PrintStmt`, `IfStmt`, entre otros.

---

## ## 6. Recuperación de errores

Modo pánico con sincronización en `;`, `}` y palabras clave de sentencia.

---

## ## 7. Decisiones de diseño

Se eligió Python con lexer y parser manuales para controlar mensajes de error y mantener el proyecto sin dependencias externas. La gramática es simple y adecuada para análisis descendente recursivo.

---

## ## 8. Ejecución

```
` ``bash
cd vec3d
py vec3d.py examples/prueba_valida.vec3d
` ``
```

---

## ## 9. Archivos de prueba

Archivo   Descripción
----- -----
`prueba_valida.vec3d`   Programa correcto
`prueba_error_sintactico.vec3d`   Falta `(` en literal vectorial
`prueba_error_semantico.vec3d`   Vector con 2 componentes